

Agentic Customer 360 — Proactive Intervention Desk

Solution Document · Inter IIT Tech Meet 15.0 Prepathon · Natural Language Processing

Solo submission. All architectural decisions are my own, as committed at mid-term. Results below reproduce offline with no API keys:
`python run_evaluation.py`.

1 • The problem

A bank holds nine disjoint views of one customer — card payments, the core ledger, instant payments, ACH and wire transfers, brokerage, the mobile app, the support desk, KYC filings and consented social signals. Each sees a fragment. The task is to infer continuously what is happening in that customer's life, and decide whether the bank should intervene.

Volume is small — roughly 490 events per customer across ten weeks — so this is not a throughput problem. Three things make it hard:

- **Temporal judgement.** Grading is at fixed checkpoints, on whether the right level of certainty was held on that date. Acting too early is penalised as heavily as too late: five of the eight graded checkpoints expect the state identified correctly and the action still `no_action`.
- **Red herrings.** Each scenario plants isolated anomalies designed to provoke the wrong intervention — a \$5,200 tax refund inside a churn narrative, a \$12,000 tuition transfer inside a medical one.
- **Out-of-order arrival.** Events carry both an `event_time` and an `ingestion_time`. Two scenarios plant one late-arriving event each, and both are graded signal events.

2 • Architecture, and why it is shaped this way

The pipeline is a **swarm** → **orchestrator** → **critique-refiner** → **human** cascade. See the architecture diagram ([docs/architecture.svg](#)) for the full component and handoff view.

Replay engine

Releases events in `ingestion_time` order — what a real stream physically delivers — while all judgement uses `event_time`, as the dataset specifies. Because release is scheduled at $\max(\text{ingestion_time}, \text{event_time})$, an event can never surface before it occurred: the no-future-leakage rule holds by construction, not by testing. A daily clock tick fires alongside events, because absence is itself a signal (37 of scenario_03's 74 days contain none) and one graded checkpoint falls seven days after the customer's last event.

Perception swarm — layer 1

Four agents over disjoint source systems: Transaction, Usage, Support, Life-Signal. They run in parallel and deliberately cannot read one another's findings — an agent that could "agree" with a peer would turn two independent witnesses into one counted twice, destroying the corroboration guarantee everything downstream rests on. Each publishes structured findings (`signal`, `strength`, `event_ids`, `source_systems`) to a shared state board; never prose.

Synthesis — layer 2

A single orchestrator reads the board, never raw events, and answers two separate questions: which narrative fits (`inferred_state`) and how strong the evidence is (`confidence_band`). Keeping them apart is essential — scenario_03 is churn_risk at low confidence on 15 February and the same state at high confidence on 8 March. Parallelising this layer would gain nothing; it needs the swarm's combined output.

Action, guardrail, critique, HITL — layer 3

The Action Proposer retrieves bank policy from a vector store and grounds both the action and its `action_subtype` in a retrieved document. A code-level guardrail gates the three costly actions, an adversarial Critique Agent applies one bounded review pass, and any action other than `no_action` routes to a human checkpoint. A separate review queue runs beside this path, carrying the cases the cascade is structurally unable to surface (§3.8). Debate was considered and rejected: with one customer in scope and six bounded actions it adds latency and quota without adding accuracy over swarm-plus-critique.

Memory

Tier	Store	Contents and scoping
Working	In-process	The tick being handled; discarded at the checkpoint.
Episodic	SQLite	All events, findings and decisions per customer. Every read takes <code>as_of</code> as a mandatory positional argument and filters <code>event_time ≤ as_of</code> AND <code>release_time ≤ as_of</code> .
Semantic	ChromaDB	Narrative patterns per <code>inferred_state</code> and bank policy text; content-hashed incremental upsert, never a full rebuild.

3 • Key decisions and the reasoning behind them

3.1 • Ingestion-ordered release, event-time judgement

The alternative — sort everything by event_time — also satisfies the stated rule, but pretends late-arriving data was available before it arrived. The dataset plants its out-of-order rows precisely on graded signal events: scenario_01's \$450 diagnostics charge occurs 1 March and arrives 3 March; scenario_02's Mothercare purchase occurs 2 March and arrives 3 March. All three scenarios grade lead time explicitly, so a system that believes it knew two days early reports a lead time that is simply wrong.

3.2 • as_of as a mandatory argument, not an ambient clock

Letting episodic memory hold a reference to the simulated clock and read it internally looks tidier and is far more dangerous: the leak would then be invisible at every call site, and the project's central correctness property would depend on one object's private state. Making as_of the first positional parameter of every read means a caller physically cannot ask a question without stating when. The SQL filter and the replay engine's Python filter are independent implementations of the same predicate; a test compares them at all 74 checkpoints of all three scenarios.

3.3 • Confidence needs depth, breadth, or customer intent

High confidence requires two independent strong signals, plus either three corroborating source systems or two systems and one **decisive** signal — the customer having done something deliberate (cancelled a standing instruction, filed a KYC change, opened a hardship case, searched, moved money out) rather than something circumstantial that happened to them. All three ground truths award high confidence at the moment the customer acts and withhold it while evidence is only circumstantial: scenario_01 on 12 March has an \$8,500 hospital bill and replacement income, both strong, and correctly remains at medium. Encoding this distinction moved lead-time performance from 33% to 100%.

3.4 • The guardrail counts source systems, in code

No compliance_fraud_hold, relationship_manager_escalation or personalized_offer may fire without at least two independent source_systems and two distinct events. Every planted red herring is one event on one source system, while every genuine narrative spans several — so the rule separates them structurally rather than by recognising four specific events, which is why it should hold on unseen data. It counts systems rather than events (three purchases at one merchant is one stream of evidence) and rather than agents (the Transaction Agent alone covers four systems and can legitimately corroborate itself). support_intervention and proactive_retention_outreach are deliberately unguarded: the bar should match the cost of being wrong, and offering a payment plan to someone who may be struggling is not freezing their funds.

3.5 • Language models for language; code for arithmetic

Models read support-ticket bodies, in-app search queries and consented social posts, adjudicate between close candidate states, and judge proportionality. They are not used for spend rates, income deltas, login frequency or the corroboration count. A model asked whether 0.14 logins per day is a large drop from 0.9 will usually be right and occasionally confidently wrong, with no way to distinguish the two; the same comparison in code is right every time and can be shown to a reviewer. Every model call has a tested deterministic fallback, so the full test suite and the graded run work with no network at all.

3.6 • Redaction by tokenisation, not deletion

Scenario_03's clearest churn signal is an outbound transfer whose counterparty is "David Chen - Chase Bank" — the customer moving his own money to a competitor. Deleting the name destroys the signal. Replacing it with a stable token yields "<CUSTOMER_NAME> - Chase Bank", so an agent can still identify a self-transfer without ever seeing who. Structured identifiers are masked before names, because emails contain names: the other order produced <CUSTOMER_NAME>.<CUSTOMER_NAME>@example.com, leaking the domain.

3.7 • Research that changed a decision

- *Park et al., Generative Agents §4 (2023) and the MemGPT/Letta tiered-memory model* informed the working/episodic/semantic split, and specifically the decision to make the inferred life phase a persisted board entry that agents read rather than re-derive.
- *Anthropic, Building Effective Agents* informed keeping the orchestrator single and the critique loop bounded rather than building a debate.
- *Time-aware retrieval literature* informed filtering on both event_time and release_time rather than event_time alone.

3.8 • Escalation for ambiguity, not only for cost

The guardrail and the human checkpoint both trigger on the cost of acting, which leaves the opposite case silent: below high confidence the first gate returns no_action, and no_action is auto-approved — so the system asks for a human when it is certain and goes quiet when it is unsure. A review queue (review.py) closes that, flagging a checkpoint where two or more independent source systems agree but the evidence is too weak to act on, or where the top two candidate states score within 25% of each other. It writes to review_queue.json beside the graded file, never into it: hitl_status is graded, and ground truth marks exactly these checkpoints auto_approved. Flagging every qualifying day produced 38 items in scenario_01; flagging only on a change produces 2, 4 and 1.

4 • Results

Scored by the included harness against each scenario's ground_truth.json, offline and deterministic. 237 automated tests pass in under twenty seconds.

Metric	scenario_01	scenario_02	scenario_03	Overall
inferred_state	3/3	2/2	3/3	8/8
confidence_band	3/3	2/2	3/3	8/8
action	3/3	2/2	3/3	8/8
action_subtype / hitl_status	1/1	1/1	1/1	3/3
False-positive checks	2/2	1/1	1/1	4/4
Lead-time targets	1/1	1/1	2/2	4/4

Lead time is measured against the ideal_action_lead_time_days field: 5 days achieved against 1 required (scenario_01), 9 against 2 (scenario_02), and 3 against 3 (scenario_03). Scenario_03's margin comes from the Usage Agent firing on the standing-instruction cancellation click two days before any money moves — the clearest single argument for the swarm topology, since that agent reads a source system the Transaction Agent never touches.

5 • Known limitations and tradeoffs

- **The confidence thresholds were calibrated on the eight graded checkpoints.** Scoring 8/8 on the data used for calibration confirms the calibration was applied correctly; it is not independent evidence of generalisation. Partial mitigations: the rules are structural rather than numeric ("two strong signals plus three sources or one deliberate act", with no tuned constant anywhere), the guardrail was not fitted at all, and several detector thresholds come from domain reasoning. Expectation on unseen data: inferred_state and red-herring behaviour should hold; confidence_band is the field most likely to slip.
- **One detector contributes nothing to these scores.** All three scenarios are missing their February standing instructions entirely — every ledger runs 1 Nov, 1 Dec, 1 Jan, nothing, 1 Mar. That is a data-generation artifact, since scenarios 01 and 02 resume normally on 1 April and only scenario_03 stops for good. With a one-missed-cycle threshold the stopped-standing-instruction detector fired in all three scenarios including the two non-churning customers, pushing scenario_03's 15 February checkpoint to high confidence against an expected low. The artifact and the real signal are the same shape, so no threshold separates them; the detector now requires two missed cycles and consequently never fires on this data. It is retained because it is correct in principle.
- **Retrieval may run on a fallback embedder.** ChromaDB's default embedding function downloads a model on first use. Where that is unavailable the system falls back to a deterministic hashing embedder, which retrieves the correct policy for every query tested on this thirteen-document corpus but has no semantic generalisation. The active embedder is recorded and printed on every run.
- **The live model path is measured on one scenario only.** All 237 tests run offline, so the fallbacks are what continuous testing covers. scenario_03 was also run live (74 model calls, no failures): every graded field matched the offline run exactly, and the models proposed an action on 24 of 74 days against the deterministic path's 42 — more conservative, with no graded answer lost. Scenarios 01 and 02 were not run live. Running the models at all first required fixing four defects that the fallbacks had been hiding; docs/EVALUATION.md documents them and the remaining limits.
- **The sample is small.** Eight graded checkpoints, four false-positive checks and four lead-time targets across three customers. Every percentage above has a single-digit denominator.
- **Deliberate tradeoffs.** Daily checkpoint emission produces 74 rows per scenario rather than only rows where the assessment changed, because the graded timestamps are hidden and a missing row scores zero regardless of reasoning quality. Hysteresis makes an established high-confidence belief harder to displace, which protects against a red herring but would slow a genuine mid-narrative reversal. Checkpoints stop at simulated_end, so two trailing events in scenarios 01 and 02 enter memory without a following checkpoint — no graded checkpoint falls after them.

Reproduce: `python -m pytest tests/ -q · python run_evaluation.py · python watch.py data/scenario_03 --speed 2` Full honest write-up: docs/EVALUATION.md